

**UNIVERSIDADE DO ESTADO DE SANTA CATARINA – UDESC
CENTRO DE EDUCAÇÃO SUPERIOR DO ALTO VALE DO ITAJAÍ – CEAVI
PROGRAMA DE GRADUAÇÃO – ENGENHARIA DE SOFTWARE**

**JOSÉ OTÁVIO ÁVILA ALVES PIRES
NATANIEL DE OLIVEIRA DA SILVA**

**CONCEITOS, TÉCNICAS E FERRAMENTAS CASE UTILIZADAS NAS
ABORDAGENS DE ENGENHARIA REVERSA E REENGENHARIA E COMO SE
APLICAM NO CONTEXTO DE PROJETO DE ARQUITETURA DE SOFTWARE**

**IBIRAMA
2025**

SUMÁRIO

1. INTRODUÇÃO.....	15
2. ENGENHARIA REVERSA.....	15
2.1. MÉTODO FUSION / RE.....	16
2.2. MÉTODO FUSION RE/I.....	16
2.3. MÉTODO DE ABORDAGEM GENÉRICA.....	16
3. REENGENHARIA.....	16
4. CONCLUSÃO.....	18
REFERÊNCIA.....	19

1. INTRODUÇÃO

O mercado de software é uma das áreas que mais sofre mudanças ao longo do tempo, os programas tendem a entrar em um estado de mudança contínua ao longo dos anos, seja por conta da evolução de linguagens, mudança de metodologia e novas técnicas que faz com que esses softwares possam se tornar obsoletos.

Segundo Osborne e Chikofsky (1990), a variedade de problemas que envolve manutenção de software cresce constantemente, sendo que as soluções não acompanham essa evolução. Tais problemas se dão por conta de código mal aplicado e principalmente projeto e documentação mal feita.

Com a crescente demanda de softwares com qualidade e bem documentados, alguns produtos mais antigos tendem a perder espaço em um mercado tão vasto, caso esses aplicativos desejam voltar à tona é necessário que sejam feitas mudanças, sejam mudanças de linguagem, funcionalidades, estruturam e até em sua documentação. É nesses cenários que a Engenharia Reversa e a Reengenharia que conseguem auxiliar a aprimorar tais softwares carentes em documentos e qualidade.

2. ENGENHARIA REVERSA

Segundo Locci (2006), a Engenharia Reversa nasceu como uma lógica que mapeia a construção de ferramentas a fim de conseguir informações sobre a análise de projetos utilizando programas já consolidados. Esse processo tende a ser um tanto quanto desafiador, pois, para garantir a sua eficácia e, ao mesmo tempo, a integridade do sistema que será gerado a partir desse processo, é necessário um grande esforço a partir da utilização de diversos recursos, como dinheiro e tempo. Graças às constantes demandas por alterações em sistemas, a documentação torna-se fundamental para mapear os processos presentes no ciclo de vida do software, o que, por sua vez, não permite a inconsistência da documentação relacionada ao produto em questão. Segundo Schneidewind (1987), é necessário colocar em prática três pilares: a compreensão, a modificação e a revalidação do sistema. Para Gall (1994), o processo da Engenharia Reversa é descrito por reconstruir a documentação de um software, a fim de obter todas, ou, ao menos, o máximo de informações necessárias para a construção do novo projeto. Ademais, para Muhammad (2005), o processo de Engenharia Reversa é baseado em analisar o código-fonte, as documentações, caso existam, e as regras de negócio do sistema. Com essas informações, é possível realizar a abstração geratriz da Engenharia Reversa. Para Locci (2006), a complexidade deste processo reside, principalmente, em manter as regras de negócio do sistema legado intactas. Dessa forma, a partir desta afirmação, é possível chegar à conclusão de que abstrair um software de forma a manter suas regras de negócio e funcionalidades principais intactas é, de certa forma, desafiador. Nesse sentido, é possível, também, pontuar algumas ferramentas CASE que auxiliam o processo da Engenharia Reversa, como:

- DR. CASE: Conjunto de ferramentas para modelagem conceitual, lógico e físico de banco de dados;
- CASE STUDIO: Extrai diversos aspectos do banco de dados, como atributos, colunas, entidades, entre outros.

- ROSE: Auxilia, principalmente, na análise de sistemas orientados a objetos, visto que utiliza modelagem de dados na linguagem UML.

Ademais, segundo Locci (2006), é possível, também, apresentar alguns métodos utilizados na Engenharia Reversa. Dentre eles, pode-se destacar, mas não somente, o método Fusion/RE, o método Fusion RE/I e o Método de Abordagem Genérica de Engenharia Reversa.

2.1. MÉTODO FUSION / RE

O método Fusion/RE baseia-se nos seguintes objetivos:

- Satisfazer, senão todas, a maioria das necessidades dos usuários, e, desta forma, melhorar a programação de sua interface principal no atendimento ou padronização de suas funções.
- E, como principal objetivo, possibilitar a transformação de um sistema procedimental para um sistema orientado a objetos, dessa forma, recuperando o projeto de sistemas legados. A possibilidade da recuperação do projeto de software está na utilização do código fonte e das entrevistas com os usuários.

2.2. MÉTODO FUSION RE/I

Derivado do método Fusion/RE, o método Fusion RE/I apresenta diversas semelhanças com seu ascendente. Segundo Locci (2006), a principal diferença destaca-se na ordenação das etapas a serem alcançadas.

2.3. MÉTODO DE ABORDAGEM GENÉRICA

De acordo com Ghannouchi et al. (1998), o método de abordagem genérica utiliza procedimentos derivados de outros métodos de Engenharia Reversa. Este reaproveitamento, por sua vez, pode destacar-se positivamente dentre os outros métodos para dirigir esse processo, visto que ele aproveita as melhores etapas de cada método para atingir os melhores resultados possíveis.

3. REENGENHARIA

Quando um sistema é grande, com diversas funcionalidades, ele pode se tornar difícil para dar manutenção, então, é aconselhável que ele seja reconstruído tendo em vista que em determinados casos pode ser mais barato do que dar a devida manutenção. Essa reconstrução deve ser feita de forma em que o software mantenha suas principais características e funcionalidade a fim de melhorar significativamente o produto sem perder seu propósito inicial. Esse processo é denominado reengenharia de software. Segundo Pressman (1995), a reengenharia, também chamada de recuperação e renovação, recuperar informações de projeto de um software existente e usa essas informações para alterar ou reconstituir o sistema, preservando as funções existentes, ao mesmo tempo em que adiciona novas funções ao software, num esforço para melhorar sua qualidade global.

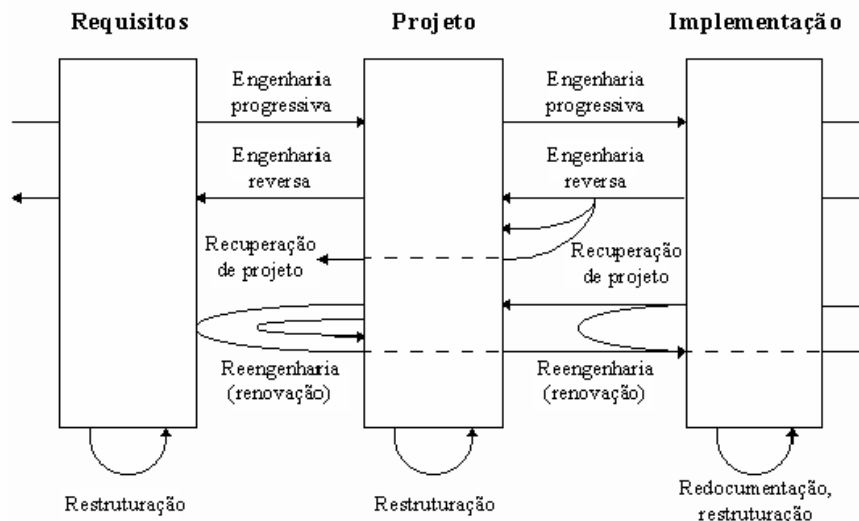


Figura 1: Ciclo de desenvolvimento de software Fonte: Chikofsky; Cross, 1990

Conforme mostrado na figura 1 a Reengenharia tem três principais etapas, a Engenharia Reversa, já explicada anteriormente, a Engenharia Progressiva sendo o processo mais comum e mais usado no desenvolvimento de software, seguindo os requisitos e documentação, e a Reestruturação.

Reestruturação é o ato de refazer alguma coisa mantendo suas principais características, muito usada no meio de software para transformar códigos mal estruturados em softwares mais limpos e entendíveis.

Segundo Quinaia (1998), o processo de reengenharia de software é constituído de duas fases distintas, a primeira "desmonta" o software objeto de reconstrução, para seu entendimento e a segunda o constrói novamente, em uma nova forma desejada. Nesse ato de reconstruir o software novamente é onde são feitas as alterações desejáveis a fim de melhorar o produto final, seja alterações de funcionalidades, linguagens e entre outras.

Existem alguns pontos que devem ser considerados para um processo de reengenharia [Warden. 1992]:

- Reengenharia deve ser executada somente se existir um argumento aceitável de custo/benefício.
- Reengenharia implica melhoria através de reprojeto.
- Reengenharia deve remover projetos ruins, mas reconhecer e manter projetos bons e simples, mesmo se ele for desestruturado.

Reengenharia é um processo significativamente bom para projetos mal estruturados ou obsoletos, porém um processo que pode se tornar caro em projetos que não precise de tantas mudanças, existem diversas outras técnicas que podem ser melhor que a Reengenharia nesse caso, como uma apenas refatoração por exemplo. Então, é de suma importância que seja feita uma análise minuciosa para aplicar a Reengenharia.

Como a reconstrução manual de sistemas é um processo lento e trabalhoso, pesquisas estão sendo feitas para a utilização de tecnologias de sistemas

transformacionais que tem como objetivo ajudar nessa reconstrução, uma das pesquisas mais avançadas nessa parte é do Draco-PUC que, de acordo com Prado (1999) o paradigma Draco é constituído em três etapas,

Uma linguagem, definida por um parser que é responsável por analisar os sistemas da linguagem e gerar a representação interna do Draco, uma Abstract Syntax Tree (AST) denominada no Draco de DAST; um prettyprinter ou unparser, que realiza a formatação da DAST, tornando-a novamente textual na linguagem do domínio; e um ou mais Transformadores, que mapeiam estruturas de uma linguagem para estruturas na mesma linguagem do domínio, chamados de transformadores Intra-Domínio.

Integrando o sistema transformacional Draco-PUC com técnicas de Fusion/RE e da ferramenta CASE Rational Rose propõe-se uma estratégia eficiente de Reengenharia de Software que é realizada em quatro passos: Segmentar, Transformar, Reprojetar e Reimplementar. Seguindo esses quatro passos pode-se chegar em um resultado satisfatório na Reengenharia.

4. CONCLUSÃO

Conclui-se, portanto, a compreensão dos termos denominados Engenharia Reversa e Reengenharia como processos imprescindíveis na reestruturação de um produto de software. Desde a fase de refazer a documentação do sistema, até a última fase da Reengenharia, o processo completo de restauração e renovação do software nota-se perspicaz e, também, desafiador. Nota-se, também, a oportunidade que esses processos trazem a sistemas legados presentes a tanto tempo em empresas que talvez possam ser muito conservadoras, e que, apesar de parecerem inúteis, a Engenharia Reversa e a Reengenharia são metodologias eficazes no que se propõem a longo prazo.

REFERÊNCIA

- PRESSMAN, R. S. Engenharia de Software. São Paulo: Makron Books, 1995.
- OSBORNE, W. M.; CHIKOFSKY, E. J. Fitting Pieces to the Maintenance Puzzle. *IEEE Software*, v. 7, n. 1, p. 14–18, 1990.
- CHIKOFSKY, E. J. e CROSS II, J. H. Reverse Engineering and Design Recovery: A Taxonomy. *IEEE Software*, v.7, n.1, pp.13-17, 1990.
- WARDEN, R. Re-engineering - A Practical Methodology With Commercial Applications. in *Applied Information Technology 12 (Software Reuse and Reverse Engineering in Practice)*. (P. A. V. Hall, ed.) - Chapman@Hall. 1992.
- DE JESUS, Elisângela Sato; FUKUDA, Ana Paula; PRADO, Antonio Francisco do. Reengenharia de Software para Plataformas Distribuídas Orientadas a Objetos. São Carlos: Universidade Federal de São Carlos, Departamento de Computação, 1999.
- LOCCI, Antonio Mateus. **Metodologia de avaliação para aquisição de uma ferramenta de engenharia reversa**. 2006. Dissertação (Mestrado em Ciência da Computação) – Faculdade de Ciências Matemáticas, da Natureza e Tecnologia da Informação, Universidade Metodista de Piracicaba, Piracicaba, 2006. Disponível em: <https://www.cliqueapostilas.com/Content/apostilas/d0f2e638e7a482d5ee74df2f486507d2.pdf>. Acesso em: 23 nov. 2025.
- MUHAMMAD, R. A. Why Teach Reverse Engineering? **Communications of the ACM**, [S. l.], jul. 2005..
- SCHNEIDEWIND, N. F. The state of software maintenance. **IEEE Transactions on Software Engineering**, [S. l.], v. 13, n. 3, p. 303-310, mar. 1987.
- GALL, H.; KLÖSCH, R.; KOFLER, E.; WÜRFL, L. Balancing in Reverse Engineering and in Object-Oriented Systems Engineering to Improve Reusability and Maintainability. In: *COMPUTER SOFTWARE AND APPLICATIONS CONFERENCE (COMPSAC)*, 18., 1994. **Proceedings [...]**. [S. l.]: IEEE, 1994.
- GHANNOUCHI, S. A.; GHEZALA, H. H.; KAMOUN, F. A Generic approach for data reverse engineering taking into account application domain knowledge. In: *EUROMICRO CONFERENCE ON SOFTWARE MAINTENANCE AND REENGINEERING (CSMR)*, 2., 1998, Florence. **Proceedings [...]**. Florence: IEEE, 1998. p. 21-28.